

RAG Pipeline

AICB-P1 · Ngày 8 · RAG

M.Sc Trần Minh Tú
Technical Director – AI Platform @ MSB

Phase 1 · Day 7 · 2026

Nội Dung Bài Học

VINUNIVERSITY

1. Retro Day 07
2. RAG — Retrieval-Augmented Generation
3. Pre-Retrieval
4. R — Retrieval
5. Post-Retrieval
6. A — Augmentation
7. G — Generation
8. Post-Generation
9. Agentic RAG
10. RAG Evaluation
11. Lab 8

Đi theo đúng thứ tự dữ liệu chạy trong pipeline: **query/question** → **R** tìm chứng cứ → **A** đóng gói → **G** sinh câu trả lời → **đo** bằng evaluation.

Mục Tiêu Ngày 8

VINUNIVERSITY

- Hiểu được tổng quan về RAG Pipeline
- Hiểu được một số pattern, module cơ bản của RAG
- Biết khi nào nên dùng RAG

Retro Day 07

01

Nhìn lại Ingestion Pipeline đã xây hôm qua, xác định điểm cần nâng cấp, và kết nối sang RAG Pipeline

Nhắc Lại: Day 07 Đã Học Gì?

VINUNIVERSITY

Nền tảng dữ liệu

- **Data strategy**: knowledge / operational / contextual data
- **Data Quality Pyramid** — chất lượng dữ liệu quyết định hơn đổi model
- **Memory architecture**: short-term vs long-term

Kỹ thuật lõi

- **Chunking**: 3 strategies + cây quyết định chọn size/overlap
- **Embedding**: cosine vs euclidean, chọn model theo trade-off
- **Vector store**: lưu gì, search gì, filter metadata ra sao
- **Ingestion pipeline** 3 khối: Input (MIME) Control → Processing → Indexing

```
Update:
-  Enrichment Pipeline
-  Compilation Pipeline
```

Nhắc Lại: Retrieval Pipeline Cơ Bản (Day 07)



Lưu ý: Pipeline Day 07 chỉ có **dense search + top-k + inject thô** vào LLM. Chưa có hybrid, rerank, augmentation có cấu trúc, citation, hay evaluation — nên "có câu trả lời" mà không biết đúng hay sai.

Ingestion Pipeline vs Retrieval Pipeline

VINUNIVERSITY

Ingestion (Offline)

- Document → Processing → Store
- Chạy batch hoặc near real-time
- Mục tiêu: chuẩn bị dữ liệu sạch, có metadata, sẵn sàng search

Retrieval (Runtime)

- Query → Search → Filter → LLM → Answer
- Chạy mỗi khi user hỏi
- Mục tiêu: tìm đúng chứng cứ, sinh câu trả lời grounded

Ingestion tạo ra index. Retrieval dùng index đó để trả lời. Chất lượng ingestion quyết định trần của retrieval.

Từ Retrieval Pipeline Sang RAG

VINUNIVERSITY

Day 07: Retrieval Pipeline

- Dense search only
- Inject context thô vào prompt
- Không có rerank, không có evaluation
- "Có câu trả lời" nhưng chưa biết đúng hay sai

Day 08: RAG Pipeline

- Hybrid search + rerank
- Augmentation: đóng gói context có cấu trúc
- Generation: grounded prompt + citation
- Evaluation: đo faithfulness, relevance, recall

Lưu ý: Day 08 nâng cấp retrieval pipeline thành **RAG pipeline** hoàn chỉnh: R (search + rerank) → A (đóng gói context) → G (sinh câu trả lời grounded).

RAG — Retrieval-Augmented Generation

02

RAG không phải chỉ là gắn thêm context; nó là sự phối hợp giữa retrieval system, augmentation layer, và generation system

Ảo Giác Của LLM (The Illusion of Knowledge)

VINUNIVERSITY

Kiến thức bị đóng băng

Knowledge Cutoff

LLM chỉbiết những gì đã xảy ra **trước ngày training**. Thông tin nội bộ hay sự kiện mới là **điểm mù**.

Bản chất xác suất

Probabilistic Nature

LLM là cỗ máy dự đoán từ tiếp theo, ưu tiên sự **trôi chảy** (fluency) hơn tính **chính xác** (factual accuracy).

Hệ quả — Hallucination

Khi thiếu dữ kiện, model sẽ tự động **"bịa"** ra thông tin trông rất logic và tự tin để **làm hài lòng** người dùng.

Lưu ý: Có thể còn nhiều lý do khác nữa và mấu chốt rằng, LLM không biết bản thân không biết. Đây là lý do chính để cần một hệ thống **cung cấp chứng cứ thực** trước khi sinh câu trả lời.

RAG Là Gì?

VINUNIVERSITY

RAG — Retrieval-Augmented Generation — Kỹ thuật kết hợp **truy xuất thông tin** (retrieval) với **sinh ngôn ngữ** (generation) để LLM trả lời dựa trên dữ liệu thực thay vì chỉ dựa vào bộ nhớ huấn luyện.

R — Retrieval

Truy xuất thông tin từ kho dữ liệu ngoài (vector store, DB, search engine).

▷ Tìm đúng chứng cứ, lọc nhiễu, xếp hạng relevance.

A — Augmentation

Tăng cường prompt: đóng gói context có cấu trúc, gắn metadata, tách evidence vs question.

▷ Giảm noise, chống "lost in the middle".

G — Generation

Sinh câu trả lời bám sát chứng cứ đã augment, kèm trích dẫn nguồn.

▷ Grounded answer, citation, self-check.

RAG = **tìm chứng cứ → đóng gói có cấu trúc → sinh câu trả lời grounded**. Kết quả có thể **trích dẫn** và **kiểm chứng**.

RAG Pipeline và các kỹ thuật chính



Tại sao lại chia nhỏ các kỹ thuật ?

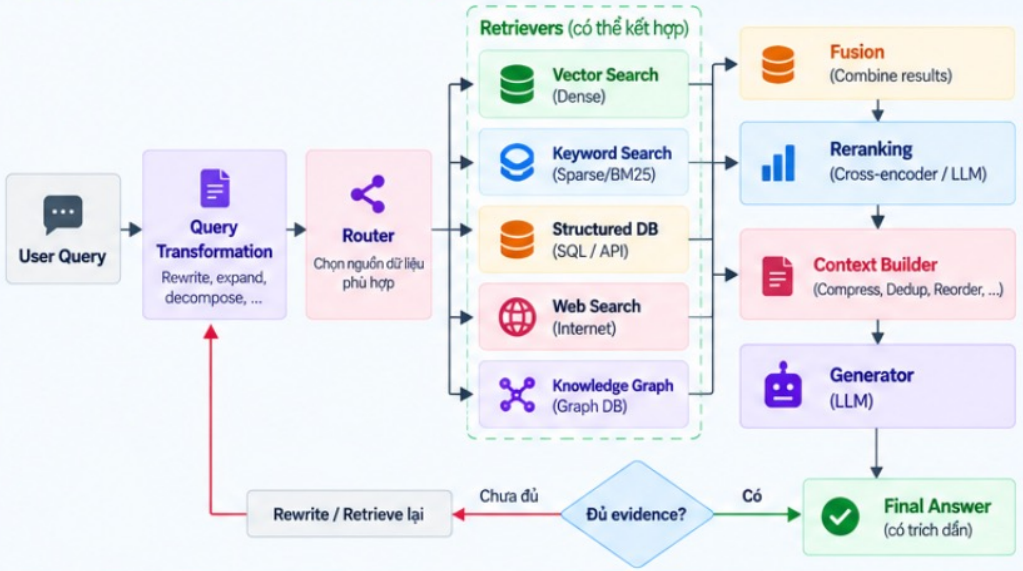
VINUNIVERSITY

Modular RAG là gì?

Modular RAG là cách tổ chức hệ thống RAG thành các module độc lập có thể kết hợp (compose), thay thế (replace), bỏ qua (skip) hoặc lập lại (loop) tùy theo đặc điểm của từng truy vấn.

- Không còn pipeline cố định.
- Linh hoạt theo nhu cầu thực tế.
- Dễ mở rộng, bảo trì và tối ưu.
- Tối ưu chất lượng và chi phí.

Kiến trúc Modular RAG (ví dụ)



Ưu điểm

Linh hoạt
Lựa chọn và kết hợp các module theo từng bài toán.

Dễ mở rộng
Thêm nguồn dữ liệu, công cụ mới mà không cần thay đổi toàn bộ hệ thống.

Tối ưu hiệu quả
Cân bằng giữa chất lượng, chi phí và độ trễ.

Dễ bảo trì
Các module độc lập, có thể nâng cấp và thay thế riêng lẻ.

Phù hợp production
Hỗ trợ các workflow phức tạp (đa nguồn, đa bước).

Các module phổ biến

Query Transformation
- Rewrite
- Query Expansion
- Multi-Query
- Step-back
- HyDE, ...

Routing
- Semantic Router
- Metadata Router
- Domain-specific
- SQL vs Vector vs Web
- ...

Retrieval
- Dense / Sparse
- Hybrid Search
- Graph Retrieval
- Structured Retrieval
- ...

Fusion
- RRF (Reciprocal Rank)
- Score Fusion
- Rank Aggregation
- Merge & Dedup
- ...

Reranking
- Cross-Encoder
- LLM Reranker
- Metadata-based
- ...

Context Engineering
- Compression
- Deduplication
- MMR (Diversity)
- Reordering
- Packing, ...

Corrective / Adaptive
- Self-RAG
- CRAG (Corrective RAG)
- Iterative retrieval
- Evidence check
- ...

Agentic (tùy chọn)
- Tool calling
- Planning
- Multi-step-retrieval
- Reflection
- ...

Vì Sao Và Khi Nào Cần RAG?

VINUNIVERSITY

Cần RAG khi

- Dữ liệu nội bộ thay đổi liên tục (policy, SOP, ticket)
- Cần câu trả lời có nguồn, kiểm chứng được
- Fine-tuning quá đắt hoặc dữ liệu quá nhạy cảm
- Cần giảm hallucination một cách có hệ thống

Chưa cần RAG khi

- Câu hỏi thuộc kiến thức chung (LLM đã biết)
- Không có corpus riêng để truy xuất
- Task không cần citation (creative writing, brainstorm)
- Prompt engineering đơn giản đã đủ

Vì Sao Doanh Nghiệp Bắt Buộc Phải Dùng RAG?

VINUNIVERSITY



Dữ liệu nội bộ = Tài sản

Policy, SOP, hợp đồng, ticket, báo cáo — LLM **không hề biết** những thứ này. Fine-tuning để nhồi facts rất đắt và dễ bị **catastrophic forgetting**. RAG = "open-book exam" — mang tài liệu vào phòng thi thay vì học thuộc lòng.



Compliance Trách nhiệm

Ngành tài chính, y tế, pháp lý — câu trả lời **phải có nguồn** và **kiểm chứng được**. Hallucination = **rủi ro pháp lý**, mất uy tín. RAG cho phép **trích dẫn** và **audit trail** cho mọi câu trả lời.



Cập nhật liên tục

Dữ liệu doanh nghiệp thay đổi **hàng ngày** (policy mới, ticket mới, sản phẩm mới). Fine-tuning lại model mỗi ngày? **Không khả thi**. RAG chỉ cần cập nhật **index** — model không cần retrain.

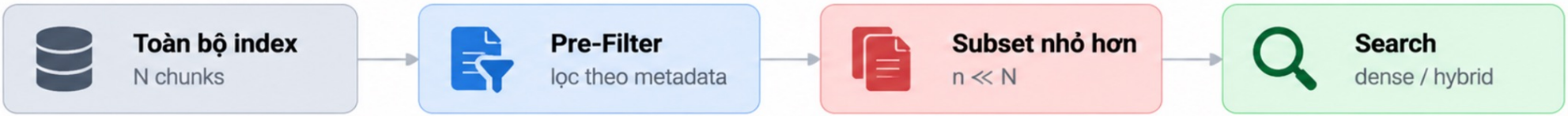
Lưu ý: RAG không phải lựa chọn — nó là **yêu cầu bắt buộc** khi doanh nghiệp cần AI trả lời dựa trên dữ liệu nội bộ, có trách nhiệm, và luôn cập nhật.

Pre-Retrieval

03

Trước khi search, phải xử lý câu hỏi và thu gọn phạm vi. Query xấu thì retrieval giỏi đến đâu cũng tìm sai.

Pre-Filtering: Thu Gọn Scope Trước Khi Search



Lọc theo metadata (Pre-Filter)

Chỉ giữ lại các tài liệu phù hợp với điều kiện nghiệp vụ trước khi chạy vector search.

Các trường metadata thường dùng

Department	Chỉ tìm trong tài liệu của phòng CS
Date	Chỉ lấy các tài liệu còn hiệu lực theo policy
Doc type	Chỉ SOP, bỏ qua FAQ
Access level	Lọc theo quyền truy cập của người dùng

Ví dụ bộ lọc

```
{
   "department": "CS",
   "date": { "gte": "2024-01-01", "lte": "2024-12-31" },
   "doc_type": ["SOP"],
   "access_level": "user"
}
```

Index lớn → search chậm và nhiều noise.
Pre-filter thu gọn scope trước khi chạy vector search, giúp **tăng tốc** và **tăng precision** cùng lúc.

Query Normalization

VINUNIVERSITY

Normalization Type	Example (User Query → Normalized Query)		What it does
Text/format normalization	" What's the capital of France??? "	→ **What is the capital of France?**	• Clean extra spaces, special characters, and inconsistent formatting. • Standardize punctuation and casing.
Spelling & typo correction	who is albert einstein born	→ who is **Albert Einstein** born	• Fix spelling mistakes and typos. • Improve query clarity.
Terminology / synonym normalization	AI applications in healthcare	→ **artificial intelligence** applications in healthcare	• Map domain-specific terms and synonyms to a canonical form. • Improve recall by using consistent terminology.
Acronym & abbreviation resolution	LLM for RAG	→ **large language model** for **retrieval-augmented generation**	• Expand acronyms and abbreviations into full forms. • Handle domain-specific short forms (e.g., LLM, RAG, NLP).
Language normalization	ai trong y tế (Vietnamese)	→ **artificial intelligence** in **healthcare** (English)	• Detect and normalize the query language (e.g., translate to target language). • Handle mixed-language queries.

Normalized Query (final)

Raw query: " ai trong y tế??? (LLM) "

Normalized: **"artificial intelligence in healthcare (large language model)"**

Benefits

- Higher recall and precision in retrieval
- Reduced ambiguity
- Better handling of diverse user inputs

Query Transformation

VINUNIVERSITY

Technique	What it does	Example (User Query → Transformed Query)	Purpose / Benefit
Rewriting & Simplification	Rewrite the query to be clearer, more grammatical, and easier to understand.	"how to instal docker on ubunto??" → **"How to install Docker on Ubuntu?"**	• Fix grammar and ambiguity • Improve retrieval quality
Query Expansion	Add related terms, synonyms, and broader/narrower concepts to the query.	"climate change" → "climate change OR global warming OR climate crisis OR greenhouse gas emissions"	• Increase recall • Cover different terminologies
Decomposition	Break a complex query into smaller sub-queries.	"What are the causes and effects of the 2008 financial crisis?" → 1. causes of 2008 financial crisis 2. effects of 2008 financial crisis	• Handle complex, multi-faceted questions • Retrieve more targeted information
Multi-query Generation	Generate multiple query variations to capture different perspectives or phrasings.	"benefits of exercise" → • benefits of physical exercise • advantages of regular workout • how exercise improves health • health benefits of fitness	• Improve recall and robustness • Capture diverse formulations
Contextualization	Incorporate conversation history or additional context to make the query self-contained.	**Previous:** "Tell me about Einstein." **Current:** "When was he born?" → **"When was Albert Einstein** born?"	• Resolve coreferences • Use context from conversation • Improve intent understanding
Translation	Translate the query to the target language (or multiple languages) for multilingual retrieval.	"trí tuệ nhân tạo trong y tế" (Vietnamese) → "artificial intelligence in healthcare" (English)	• Enable cross-lingual retrieval • Access non-English knowledge sources
HyDE / Query2Doc	Generate a hypothetical answer or document for the query, then use it as the search query.	"What is a RAG system?" → **Hypothetical document (excerpt):** *"A Retrieval-Augmented Generation (RAG) system combines information retrieval with large language models to generate ..."*	• Provide richer semantic context • Improve retrieval for abstract or underspecified queries

R — Retrieval

04

Tìm đúng chứng cứ là bước quan trọng nhất; search sai thì prompt đẹp đến đâu cũng khó cứu

Hai Trường Phái Tìm Kiếm Cốt Lõi





Dense Vector (Semantic)

Vector **ngắn, đặc** (768–1536 chiều), mọi chiều đều có giá trị $\neq 0$.

Tại sao gọi Dense? Vì embedding model nén **toàn bộ ngữ nghĩa** vào một vector dày đặc thông tin.

+ Hiểu nghĩa, paraphrase, cross-lingual
+ "hoàn tiền" $\approx$ "refund"
– Bỏ lỡ keyword chính xác (mã lỗi, tên riêng)
– Phụ thuộc chất lượng embedding model

Sparse Vector (BM25 / TF-IDF)

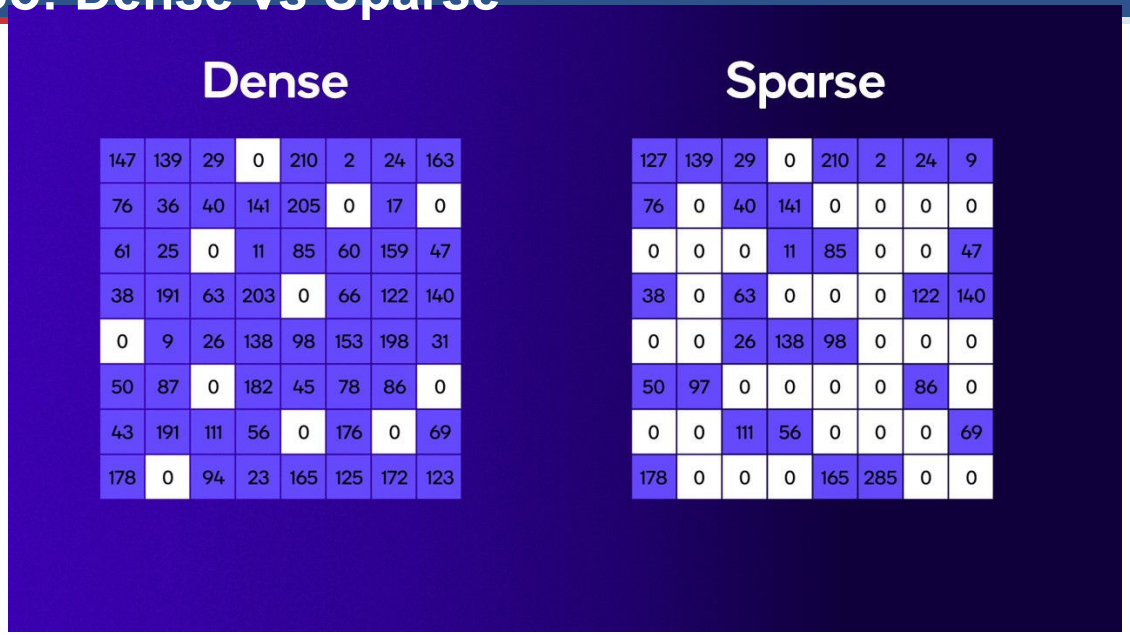
Vector **rất dài** (= kích thước từđiển) nhưng hầu hết chiều $= 0$.

Tại sao gọi Sparse? Vì chỉ các từ xuất hiện trong document mới có giá trị, phần còn lại là **số 0** (thưa thớt).

+ Match chính xác từ khóa, mã lỗi, tên riêng
+ Nhanh, không cần GPU
– Không hiểu đồng nghĩa, paraphrase
– "hoàn tiền" $\neq$ "refund"

Lưu ý: Không bên nào hoàn hảo. Dense mạnh về **ngữ nghĩa**, Sparse mạnh về **từ khóa chính xác**. Giải pháp: kết hợp cả hai → **Hybrid Search**.

Nhìn Vào Con Số: Dense vs Sparse



Dense — gần như không có ô trắng
Mỗi chiều là một **nét ngữ nghĩa** do model học ra, không map với từ cụ thể nào. Vector ngắn (768–1536) nhưng **ô nào cũng mang thông tin**.

Sparse — phần lớn là ô trắng ($= 0$)
Mỗi chiều là **một từ trong từ điển**. Document chỉ chứa vài trăm từ trên tổng 50.000 từ → đại đa số chiều bằng 0.

> **Ô trắng** $= 0 =$ **không có tín hiệu** — đó là lý do một bên gọi "đặc", bên kia "thưa".

Semantic Search (Dense Vector)



768–1536 chiều, mọi ô $\neq 0$

Cách hoạt động

- Query và document đều được **embed** thành dense vector
- So sánh bằng **cosine similarity** hoặc dot product
- Hiểu được **nghĩa**, paraphrase, cross-lingual

Giới hạn

- Bỏ lỡ **keyword chính xác**: mã lỗi, tên riêng, số ticket
- Phụ thuộc vào chất lượng embedding model
- Không phân biệt được từ đồng âm khác nghĩa

VD thất bại: Query "lỗi **ERR-4012**" → trả về docs "lỗi hệ thống chung" thay vì đúng mã lỗi.

Corpus nhiều câu tự nhiên, FAQ, knowledge base. Dense search là baseline tốt nhưng **không đủ cho production**.

Lexical Search (Sparse Vector — BM25)



= số từ trong từ điển, hầu hết = 0

Cách hoạt động

- Đếm tần suất từ (TF-IDF / BM25)
- Tạo **sparse vector**: hầu hết chiều = 0
- Match chính xác **từng từ**, không cần hiểu nghĩa

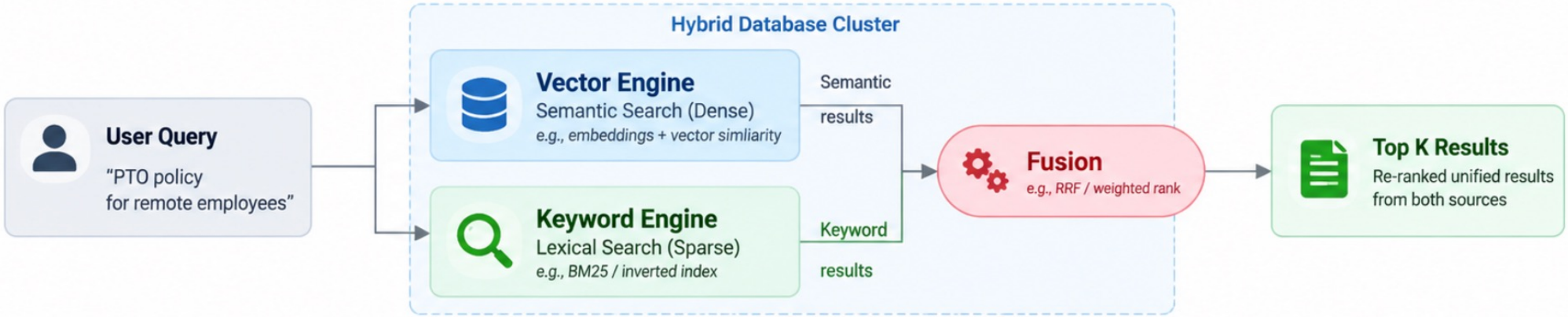
Giới hạn

- Bỏ lỡ **paraphrase** và đồng nghĩa
- "hoàn tiền" vs "refund" → không match
- Không hiểu ngữ cảnh, chỉ đếm từ

VD thất bại: Query "**nghỉ phép**" → bỏ lỡ doc chỉ chứa "annual leave", "PTO".

Policy, code, luật, log, mã ticket — nơi **exact term** quan trọng hơn nghĩa.

Hybrid Search



Ý tưởng

1. **Chạy cả hai search song song trên cùng query**
   Tìm kiếm theo nghĩa (dense) và theo từ khóa (sparse).
2. **Gộp kết quả bằng thuật toán fusion**
   Ví dụ: Reciprocal Rank Fusion (RRF), weighted ranking, v.v.
3. **Giữ cả nghĩa (dense) lẫn keyword (sparse)**
   Kết hợp điểm mạnh của cả hai để tăng độ bao phủ và độ chính xác.

Vì sao cần?

Bù trừ điểm yếu của nhau
Dense có thể bỏ lỡ từ khóa cụ thể (ví dụ: "ERR-4012"),
Sparse có thể bỏ lỡ truy vấn diễn đạt khác (ví dụ: "nghỉ phép" = "PTO").

Tăng độ bao phủ và độ chính xác
Kết quả đa dạng hơn, phù hợp cả truy vấn tự nhiên lẫn truy vấn chính xác.

Hiệu quả thực tế
Đặc biệt hữu ích cho corpus có cả ngôn ngữ tự nhiên, tên riêng, mã lỗi, điều khoản, v.v.

Hai Cách Gộp Kết Quả: RRF vs Alpha Weighting

VINUNIVERSITY

RRF (Reciprocal Rank Fusion)

$$\mathrm{RRF}(d) = \sum_{r} \frac{1}{k + \mathrm{rank}_r(d)}$$

- Chỉ dùng **thứ hạng**, bỏ qua điểm số
- k thường được chọn mặc định là 60

Alpha Weighting (Score Fusion)

$$\mathrm{score}(d) = \alpha \cdot s_{\mathrm{dense}} + (1-\alpha) \cdot s_{\mathrm{sparse}}$$

- Dùng **điểm số** trực tiếp đã được normalize về cùng scale
- **Cần tuning** α theo domain
- Điểm khởi đầu — FAQ: $\alpha \approx 0.7$–0.9; code, log, luật: $\alpha \approx 0.2$–0.4

Mới bắt đầu → **RRF** (đơn giản, không cần tuning).
Cần kiểm soát tỷ trọng dense/sparse → **Alpha weighting**.

Ba Loại Corpus, Ba Kết Quả Khác Nhau

VINUNIVERSITY

KB / Domain	Đặc tính dữ liệu	Thách thức	Loại embedding phù hợp	Mức độ đưa vào context	Ghi chú
news (tin tức)	Văn xuôi, câu tự nhiên, thay đổi liên tục, thông tin thời sự	Paraphrase, nhiều bài báo nói về cùng một sự kiện	**Dense** (semantic)	**Thường không** (dài, liên tục)	• Nên lọc theo thời gian, nguồn uy tín • Chunk theo đoạn/section
law (luật, quy định)	Cấu trúc chặt chẽ, Điều/Khoản, thuật ngữ pháp lý, ổn định	Dễ nhầm nghĩa số, điều khoản, ngoại lệ	**Sparse** (lexical / keyword)	**Thường không** (quá dài)	• Nên parse cấu trúc (Điều, Khoản) • Kết hợp keyword + semantic • Lưu metadata (số điều, hiệu lực, ...)
cards (hồ sơ, profile)	Nhiều tên riêng, con số, các thực thể mô tả gần giống nhau	Nhiều bản ghi giống nhau, dễ nhầm lẫn	**Hybrid** (dense + sparse)	**Có** (ngắn, ổn định)	• Chunk theo trường thông tin • Dùng metadata (ID, loại, thời gian) • Phù hợp cho truy vấn tra cứu chính xác

Kết luận

- **Không có một cấu hình RAG** tối ưu cho **mọi loại dữ liệu**.
- Cần **hiểu đặc tính dữ liệu** (cấu trúc, độ dài, tần suất thay đổi, mức độ trùng lặp) để chọn cách chunking, embedding và chiến lược đưa vào context phù hợp.
- **Bản chất corpus quyết định hiệu quả RAG** — nên thiết kế pipeline và cấu hình riêng cho từng domain.

Tuning α: Đừng Đoán, Hãy Đo

VINUNIVERSITY

Quy trình 4 bước

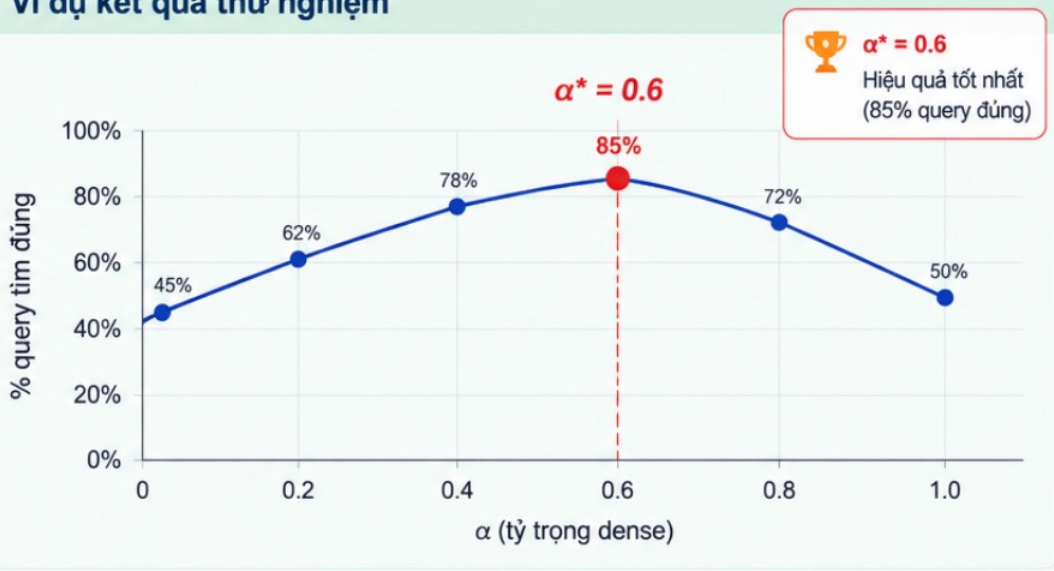
1 — Chuẩn bị test set
Chọn 15–20 query đại diện + chunk đúng (ground truth) để đánh giá.

2 — Quét giá trị α
Thử các giá trị α ∈ {0, 0.2, 0.4, 0.6, 0.8, 1.0} bằng grid search.

3 — Đánh giá kết quả
Với mỗi α, đếm bao nhiêu query lấy được chunk đúng trong top-k (ví dụ: k = 5).

4 — Chọn α tốt nhất
Chọn giá trị α có hiệu quả cao nhất, sau đó tinh chỉnh quanh giá trị đó (±0.1).

Ví dụ kết quả thử nghiệm



Ví dụ với một số truy vấn

#	Query	Top-1 result (α = 0, sparse-only)	Top-1 result (α = 0.6, hybrid)	Top-1 result (α = 1, dense-only)	Ground truth	Nhận xét
1	pension scheme eligibility	Keyword match (FAQ)	**Policy document (correct)**	Similar but not exact	Policy document	Hybrid tốt hơn
2	remote working policy	Exact keyword match	**Policy document (correct)**	Unrelated (dense miss)	Policy document	Dense-only bỏ lỡ
3	health insurance for family	Unrelated	**Relevant document (correct)**	Related but not specific	Benefits guide	Hybrid kết hợp tốt
4	PTO meaning	Exact keyword match	**Glossary document (correct)**	Related (less precise)	Glossary document	Cả hai đều tốt

Post-Retrieval

Retrieval trả về rất nhiều kết quả, có kết quả trùng lặp, có kết quả mâu thuẫn, v.v

05

Reranking: Vì Sao Top-K Chưa Đủ?

VINUNIVERSITY

1. Vấn đề: Retriever "thô và rộng"

- Retriever (vector + keyword) có thể truy xuất **hàng triệu** tài liệu cực nhanh, nhưng đánh giá sự liên quan chỉ **sơ bộ**.
- Kết quả trả về có thể chứa nhiều tài liệu chưa thực sự phù hợp.
- Tài liệu đúng nhất có thể nằm ở vị trí thấp (ví dụ: Top 10), không phải Top 1.

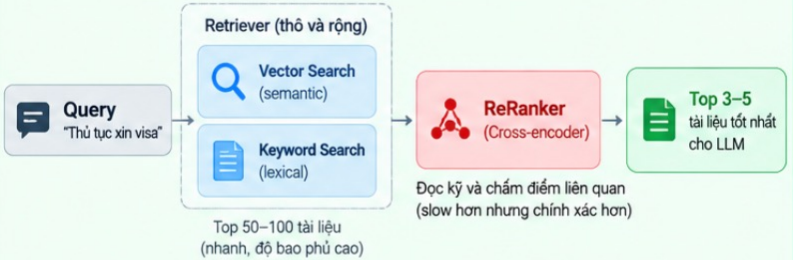
Ví dụ

Query: "Thủ tục xin visa"

Top 1	Giá làm visa
Top 2	Lịch sử visa
...	
Top 8	Các bước làm thủ tục xin visa ← đúng!
...	

2. Giải pháp: Retrieve-and-Rerank

- **Giai đoạn 1**: Search rộng → lấy Top 50–100 tài liệu nhanh, đa dạng.
- **Giai đoạn 2**: Dùng Cross-encoder (re-ranker) đọc kỹ query + document, chấm điểm liên quan, sắp xếp lại → chọn Top 3–5 cho LLM.



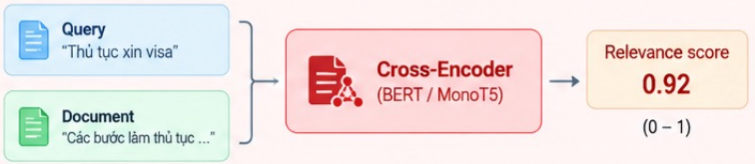
Semantic Search (Bi-Encoder)

- Bi-Encoder mã hóa Query và Document thành vector **riêng biệt** trong cùng một không gian.
- Tính độ tương đồng (cosine similarity) để tìm các tài liệu gần nhất.
- Cực **nhanh**, phù hợp để truy xuất số lượng lớn tài liệu.



Cross-Encoder (Reranker)

- Query + Document được ghép thành **1 input**, model đọc cùng lúc (ví dụ: BERT, MonoT5, ...).
- Đưa ra điểm liên quan chính xác hơn (hiểu ngữ cảnh, ý nghĩa sâu).
- Chậm hơn Bi-Encoder → chỉ dùng cho danh sách nhỏ (ví dụ: Top 50–100).



Lưu ý: Dùng kết hợp **top-k**, **ngưỡng similarity**, và có/**không rerank**. Mọi hyperparameter của RAG (k, threshold, model, ...) nên được chọn bằng test set, không bằng cảm giác.

MMR: Chống Trùng Lặp Trong Context

VINUNIVERSITY

Vấn đề: Redundancy

- Re-ranker thường đưa các chunk liên quan nhất lên đầu, nhưng nếu các chunk này **nói cùng một ý** thì kết quả kém đa **dạng**.
- LLM sẽ tốn **token** cho các nội dung lặp lại, không có thêm thông tin mới.

Ví dụ

Query: "Thời gian xử lý hoàn tiền là bao lâu?"

Chunk 1	"Hoàn tiền mất 7 ngày."
Chunk 2	"Tiền sẽ được hoàn sau 7 ngày."
Chunk 3	"Thời gian xử lý hoàn tiền là 7 ngày."

Cùng một ý (nội dung trùng lặp)

Giải pháp: Maximum Marginal Relevance (MMR)

Chọn các chunk vừa **liên** quan với query, vừa **khác biệt** với những chunk đã chọn.

$$\text{MMR} = \text{Sim}(d, q) - \lambda \cdot \text{Sim}(d, d_{\text{selected}})$$

- Độ liên quan với query (càng cao càng tốt)
- Độ giống với các chunk đã chọn (càng thấp càng tốt)

- **d**: candidate chunk
- **q**: query
- d_{selected}: các chunk đã chọn
- λ: tham số cân bằng ($0 \le \lambda \le 1$)

Các bước

1. Chọn chunk liên quan nhất với query (giống re-rank thông thường).
2. Với mỗi chunk tiếp theo, tính MMR (ưu tiên chunk vừa liên quan, vừa khác biệt).
3. Lặp lại cho đến khi lấy đủ Top-k.

Ví dụ so sánh kết quả

Top-3 từ Re-ranker (chưa dùng MMR)

1. "Hoàn tiền mất 7 ngày."
2. "Tiền sẽ được hoàn sau 7 ngày."
3. "Thời gian xử lý hoàn tiền là 7 ngày."

✗ Các chunk trùng nội dung, kém đa dạng.

Top-3 từ MMR (λ = 0.5)

1. "Hoàn tiền mất 7 ngày." — Trả lời trực tiếp
2. "Các trường hợp được hoàn tiền." — Thông tin bổ sung
3. "Quy trình và điều kiện hoàn tiền." — Ngữ cảnh liên quan

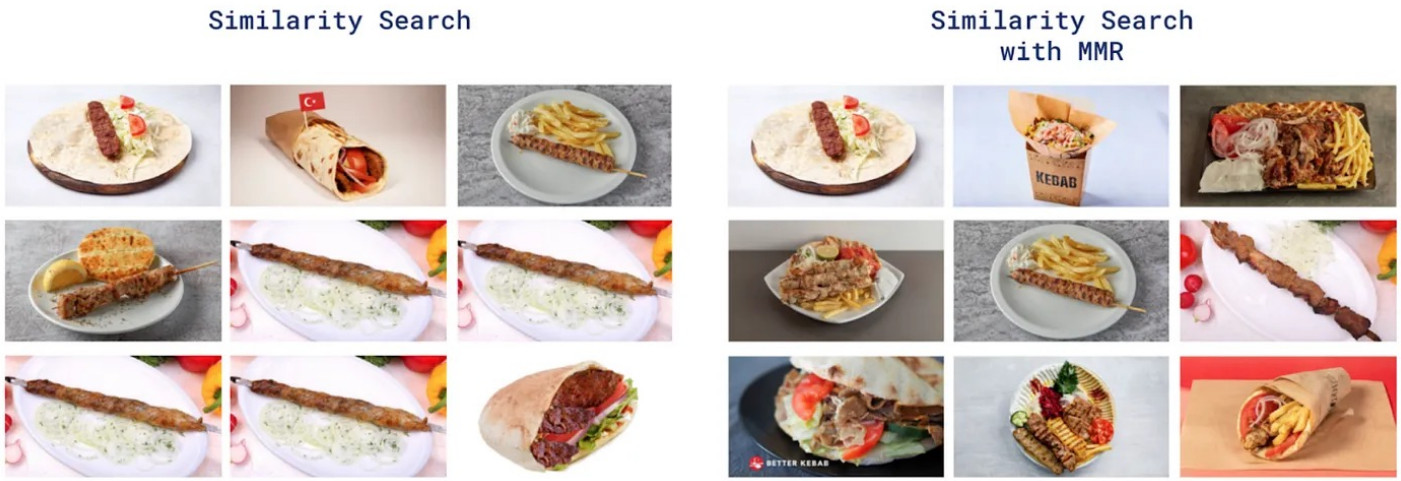
✓ Nội dung đa dạng, bổ sung thông tin cho LLM.

Kết quả cho LLM (Augmentation)

- **Chunk 1:** Thời gian xử lý hoàn tiền (7 ngày)
- **Chunk 2:** Trường hợp được hoàn tiền
- **Chunk 3:** Quy trình và điều kiện hoàn tiền

LLM có đủ và đa dạng thông tin để tạo câu trả lời tốt hơn.

MMR Trực Quan: Tìm “Kebab”



Bên trái — Similarity Search
Top 9 đều là kebab xiên nướng, có ảnh **gần như trùng nhau**. Điểm similarity rất cao, nhưng người dùng chỉ thấy **một loại duy nhất**.

Bên phải — Similarity + MMR
Vẫn là kebab, nhưng **đa dạng**: xiên, cuộn bánh, hộp, đĩa kèm khoai. Cùng độ liên quan, **nhiều dữ kiện hơn**.

> Đổi “ảnh kebab” thành “chunk hoàn tiền”: 9 chunk giống nhau **tốn token mà không thêm thông tin**. MMR đổi chút similarity để lấy **độ phủ**.

A — Augmentation

06

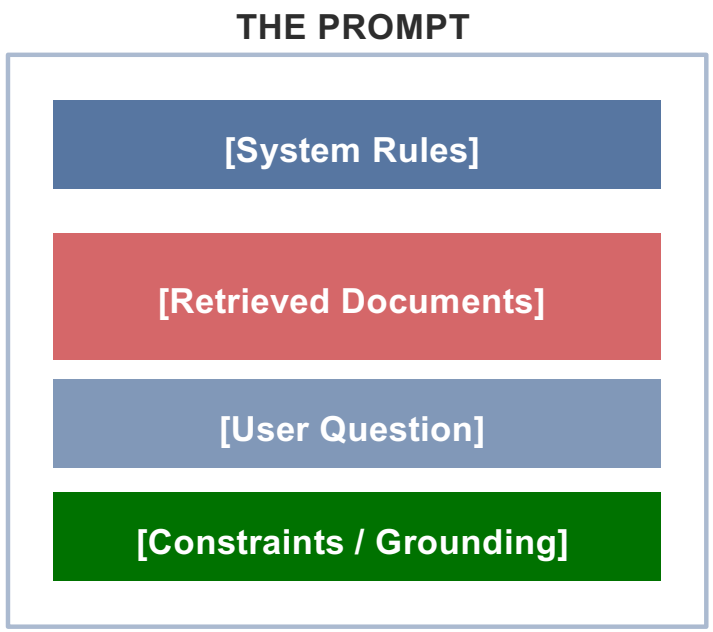
Retrieval tìm chunk. Augmentation đóng gói chunk thành context có cấu trúc trước khi đưa vào LLM.

Context Injection Là Gì?

Kỹ thuật **sắp xếp và định dạng** các chunk dữ liệu (đã retrieve) vào trong Context Window của LLM để nó **dễ đọc, dễ hiểu nhất**.

Không chỉ là nối chuỗi (`text1 + text2`).

Cách bạn định dạng quyết định việc LLM có tôn trọng dữ liệu đó hay không.



Cấu Trúc Prompt: Tách Rõ System / Context / Question

Viết chỉ dẫn rõ ràng để LLM biết:

- Đâu là "sự thật khách quan" (Context / Evidence)
- Đâu là câu hỏi của người dùng (Question)
- Đâu là instruction cho model (System)

Dùng **XML tags**, **Markdown headers**, hoặc **delimiters** rõ ràng để tách các phần.

```
<system>
Answer only from the context below.
Cite [doc_id] when possible.
</system>

<context>
[1] policy/refund-v4.pdf | Section 3
Yeu cau hoan tien trong 7 ngay...
</context>

<question>
Chinh sach hoan tien la gi?
</question>
```

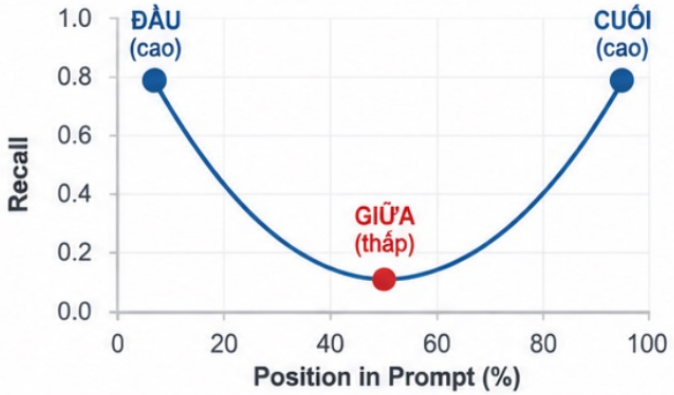
Lost in the Middle & Document Reordering

VINUNIVERSITY

Hiện tượng "Lost in the Middle"

- LLM có xu hướng chú ý nhiều hơn đến thông tin ở **đầu** và **cuối** prompt.
- Thông tin **ở giữa** dễ **bị bỏ qua**, ngay cả khi rất liên quan.
- Khi các chunk được đưa vào prompt một cách ngẫu nhiên, chunk quan trọng có thể nằm ở giữa → **RAG cho kết quả kém.**

Recall Performance theo vị trí



Ví dụ Document Reordering

Block 1 (Top 1)	Tổng quan về chính sách …	ĐẦU
Block 3	Quy trình nộp hồ sơ …	
Block 5	Điều kiện đủ để xin visa …	GIỮA
Block 4	Lệ phí và thời gian xử lý …	
Block 2 (Top 2)	Danh sách giấy tờ cần chuẩn bị …	CUỐI

Ví dụ ảnh hưởng đến kết quả

✗ Thứ tự chunk ngẫu nhiên (chunk quan trọng ở giữa)

[Block 1] Giới thiệu về chương trình visa …
[Block 3] Quy trình nộp hồ sơ …
[Block 5] Điều kiện đủ để xin visa … ← (nằm ở giữa, dễ bị bỏ qua)
[Block 4] Lệ phí và thời gian xử lý …
[Block 2] Danh sách giấy tờ cần chuẩn bị …

Kết quả LLM

"Để xin visa, bạn cần nộp hồ sơ theo quy trình …"

(Thiếu điều kiện quan trọng)

✓ Thứ tự chunk sau khi reorder (chunk quan trọng ở đầu/cuối)

[Block 5] Điều kiện đủ để xin visa … ← (đưa lên đầu)
[Block 3] Quy trình nộp hồ sơ …
[Block 4] Lệ phí và thời gian xử lý …
[Block 1] Giới thiệu về chương trình visa …
[Block 2] Danh sách giấy tờ cần chuẩn bị … ← (đưa xuống cuối)

Kết quả LLM

"Để xin visa, bạn cần đáp ứng các điều kiện sau: …
(chi tiết, đầy đủ)."

✓ Đúng và đầy đủ hơn

Metadata Integration & Conflict Resolution

VINUNIVERSITY

Metadata đi kèm mỗi chunk

Đưa **nguồn, ngày, phòng ban, version** vào context — không chỉ để cite, mà để LLM **tự xử lý mâu thuẫn**.

```
[2] hr/leave-policy-v3.pdf | 2026-01-10 | HR
Nhan vien duoc nghi 14 ngay phep/nam...
```

Rule ưu tiên đặt trong prompt

```
``If documents conflict, prefer the most recent by
date. If still ambiguous, list both and state the
conflict explicitly.''
```

Ba chiến lược khi context mâu thuẫn

1. Recency wins — ưu tiên ngày mới nhất.
Phù hợp: policy, giá, SOP — bản mới thay bản cũ.

2. Authority wins — ưu tiên nguồn có thẩm quyền cao hơn (văn bản chính thức > email > wiki).
Phù hợp: pháp lý, compliance.

3. Surface the conflict — trảcảhai và **nói rõ là mâu thuẫn**, để người dùng quyết định.
Phù hợp: khi sai một chiều gây hậu quả lớn hơn là bắt user tự đọc.

Lưu ý: Lỗi hay gặp: chunk không mang ngày tháng → LLM **không có cách nào** biết bản nào mới. Đây là lỗi **Indexing** (Day 07), lộ ra ở Augmentation.

Access Control: Retrieval Phải Tôn Trọng Quyền

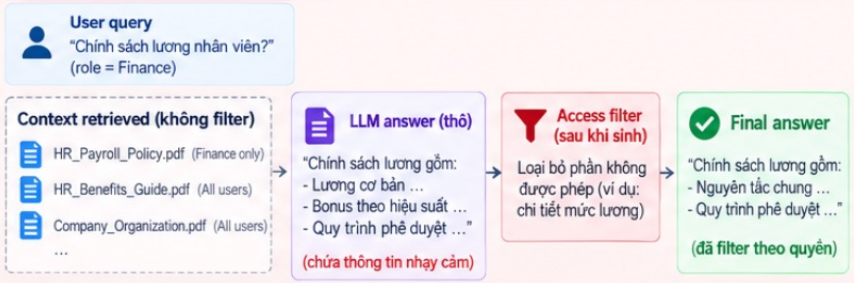
VINUNIVERSITY

1. Lọc sau khi sinh (Post-generation filtering)

Retrieve toàn bộ tài liệu → LLM sinh câu trả lời → kiểm tra quyền truy cập trên nội dung sinh ra → chỉ trả về phần được phép.



Ví dụ



Ưu / Nhược điểm

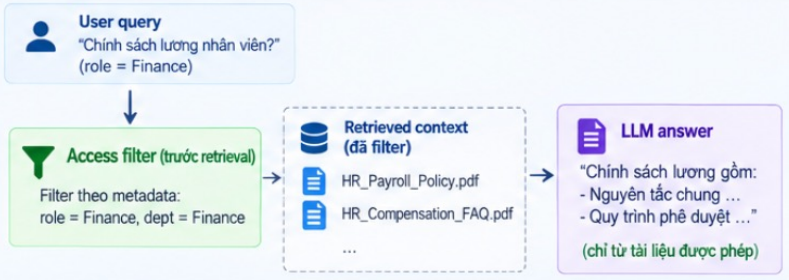
- **Ưu điểm:** đơn giản, không cần tách index theo quyền.
- **Nhược điểm:** có thể sinh ra nội dung không được phép (rủi ro rò rỉ), tốn token, chậm hơn.

2. Lọc trước khi retrieval (Pre-retrieval filtering)

Áp bộ lọc theo quyền truy cập (role, department, ACL) ngay trong truy vấn để chỉ tìm kiếm trên tập tài liệu được phép → LLM sinh câu trả lời.



Ví dụ



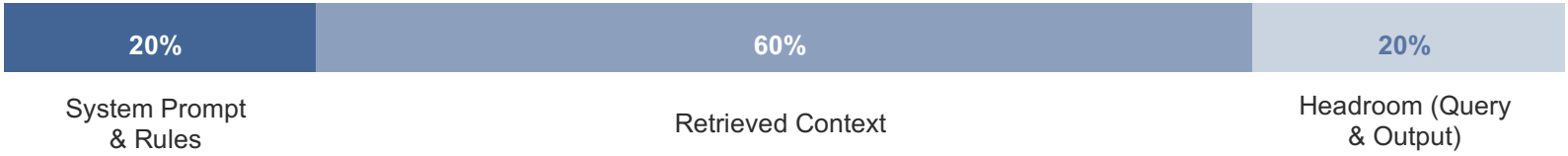
Ưu / Nhược điểm

- **Ưu điểm:** an toàn hơn, tránh đưa thông tin ngoài phạm vi vào context, nhanh hơn, tiết kiệm token.
- **Nhược điểm:** cần duy trì metadata/ACL và triển khai filter trong hệ thống tìm kiếm.

Nguyên tắc chung

- **Metadata** quyền truy cập (role, dept, classification, …) phải được gán từ đầu (ingestion).
- Chọn cách lọc phù hợp với yêu cầu bảo mật, hiệu năng và độ phức tạp của hệ thống.
- Có thể kết hợp cả hai: lọc trước để thu hẹp phạm vi, và kiểm tra lại sau khi sinh để đảm bảo an toàn tuyệt đối.

Token Budget: Context Window Là Chiếc Bánh Có Hạn



Context window có giới hạn. Nếu context chiếm quá nhiều → model không còn chỗ cho instruction và output → chất lượng giảm.

Rule of thumb: context ≤ **60%** token budget. Luôn giữ headroom cho query và output.

> Nếu context vẫn quá dài sau khi chọn top-K, có thể dùng **Context Compression** — model nhỏ nén chunk thô, chỉ giữ ý chính. Tools: *LLMLingua, Recomp*.

G — Generation

06

Augmentation đóng gói context. Generation sinh câu trả lời grounded từ context đã đóng gói.

LLM Selection & Routing

VINUNIVERSITY

Frontier / reasoning tier

Suy luận nhiều bước, so sánh nhiều nguồn mâu thuẫn, tổng hợp dài.

Chi phí và latency cao nhất.

▷ Internal tools, low-volume, câu hỏi phân tích.

Mid tier (mặc định)

Đủ tốt cho **đa số** câu hỏi RAG: đọc context, trả lời có citation.

Cân bằng chất lượng / cost / latency.

▷ Nên là **lựa chọn khởi điểm**, không phải frontier.

Small / self-hosted

Lookup trực tiếp, extract, classify. Hoặc **bắt buộc on-premise** vì dữ liệu nhạy cảm.

Cost/latency thấp nhất.

▷ High-volume, data-sensitive.

Routing — không phải mọi query đều cần cùng một model

Classify query trước → lookup đơn giản đi **small tier**, câu hỏi so sánh/multi-hop đi **tier cao**. Tiết kiệm phần lớn cost mà không giảm chất lượng ở nơi cần.

Chọn theo **độ phức tạp câu hỏi**, **bảo mật**, **ngân sách** — và **đo bằng eval**, không chọn theo tên model.

Grounded Generation

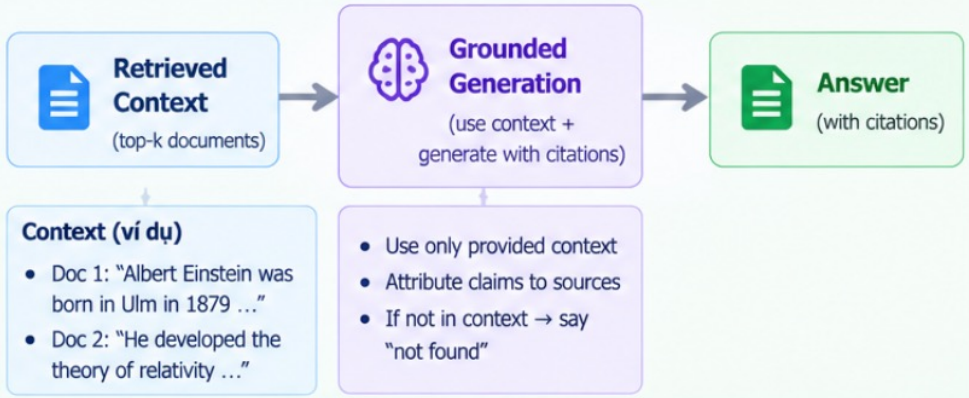
VINUNIVERSITY

1. Ý tưởng

- **Context-grounded / evidence-conditioned generation**
  - LLM chỉ sử dụng thông tin trong ngữ cảnh được cung cấp.
  - Giảm hiện tượng "bịa đặt" (hallucination).
- **Source-aware / citation-aware generation**
  - Gắn dẫn chứng (source, citation) cho từng phần thông tin.
  - Giúp người dùng kiểm chứng và truy vết nguồn.

Trả lời dựa trên bằng chứng, không dựa trên kiến thức nội tại của mô hình.

2. Quy trình minh họa



3. Ví dụ minh họa

Query: "Albert Einstein sinh năm bao nhiêu và ở đâu?"

Retrieved context (rút gọn):

1. "Albert Einstein was born in Ulm in 1879." (Source: Britannica, 2023)
2. "Einstein later developed the theory of relativity …" (Source: Stanford Encyclopedia of Philosophy, 2022)

4. Kết quả sinh có dẫn chứng

Albert Einstein sinh ngày 14 tháng 3 năm 1879 tại Ulm, Đức [1]. Ông sau đó phát triển thuyết tương đối, một trong những đóng góp quan trọng nhất của ông cho khoa học [2].

Nguồn tham khảo:

1. Britannica. (2023). Albert Einstein. https://www.britannica.com/biography/Albert-Einstein
2. Stanford Encyclopedia of Philosophy, (2022). Einstein and the theory of relativity. https://plato.stanford.edu/entries/einstein/

Grounded Generation

Prompt Construction – Generation Contract

```
Use only the provided evidence.

For every factual claim:
- Add the supporting source ID: [S1], [S2], ...
- Multiple sources may be cited: [S1][S2]
- Do not cite a source that does not support the claim.
- If the evidence is insufficient, say:
  "Not enough information in the provided sources."
```

Result

```
[S1]
source: Britannica
title: Albert Einstein
content: Albert Einstein was born in Ulm in 1879.

[S2]
source: Stanford Encyclopedia of Philosophy
title: Einstein's Philosophy of Science
content: Einstein developed the theory of relativity...
```

Post-Generation

07

Sau Generation, Câu trả lời đã được sinh ra, Post Generation dùng để kiểm tra, sửa và kiểm soát chất lượng output trước khi trả cho user

Output Control

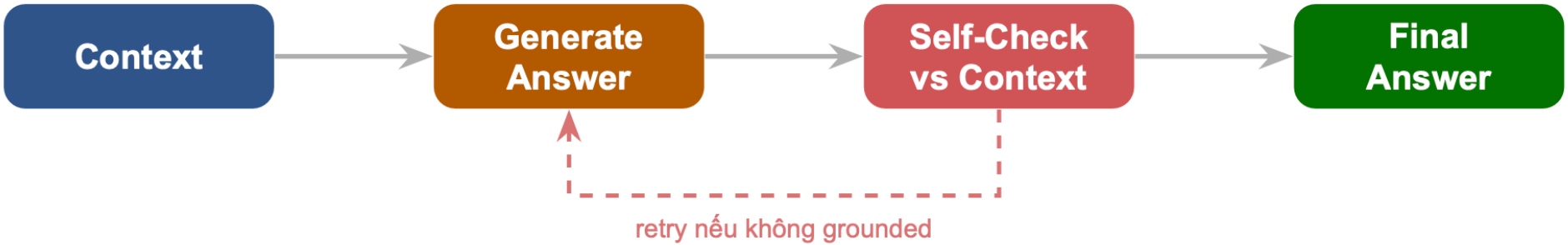
Formatting

- Yêu cầu output dạng **Markdown**, **Table**, hoặc **JSON**
- Phục vụ tích hợp vào hệ thống khác (API, dashboard)
- Dùng `response_format` hoặc instruction rõ ràng

Safety & Alignment

- Đảm bảo câu trả lời **không vi phạm chính sách**
- Không rò rỉ dữ liệu nhạy cảm không mong muốn
- Kiểm tra PII, access level trước khi trả về user

Self-Correction (Vòng Lặp Tự Sửa)



LLM tự đánh giá câu trả lời của chính mình so với context **trước khi** hiển thị cho người dùng. Nếu phát hiện suy diễn vượt chứng cứ → retry hoặc abstain.

Grounding & Abstention

VINUNIVERSITY

1. Strict Grounding (Context-only)

Chỉ sử dụng thông tin trong context được cung cấp. Cấm dùng kiến thức từ training.

- LLM phải bám sát context; không được suy đoán từ kiến thức nội tại.
- Nếu context không đủ thông tin → trả lời "Không có thông tin trong tài liệu đã cung cấp."
- Phù hợp cho các trường hợp yêu cầu độ chính xác cao (ví dụ: tài liệu nội bộ, chính sách, quy định).

Ví dụ

User "Lương nhân viên thử việc là bao nhiêu?"

Context "Chính sách nhân sự không chứa thông tin về lương thử việc ..."

LLM (Strict Grounding) "Không có thông tin về lương thử việc trong tài liệu đã cung cấp."

2. Graceful Degradation (Abstention & Guidance)

Khi context không đủ để trả lời, LLM không bịa đặt mà từ chối một cách lịch sự, đồng thời gợi ý bước tiếp theo cho người dùng.

- Nhận biết giới hạn của context, tránh suy đoán mơ hồ.
- Trả lời rõ ràng "Không biết" / "Không đủ thông tin", kèm gợi ý:
  - Làm rõ câu hỏi.
  - Cung cấp thêm tài liệu hoặc thông tin liên quan.
  - Liên hệ bộ phận phù hợp (ví dụ: HR, IT, ...).
- Giúp trải nghiệm người dùng tốt hơn, giảm rủi ro thông tin sai lệch.

Ví dụ

User "Quy trình xin visa cho nhân viên nước ngoài như thế nào?"

Context "Tài liệu hiện có chỉ đề cập đến quy trình xin visa cho khách du lịch, không có thông tin cho nhân viên ..."

LLM (Graceful Degradation) "Hiện tại tôi chưa tìm thấy thông tin về quy trình xin visa cho nhân viên nước ngoài trong tài liệu đã cung cấp. Bạn có thể thử tìm với các từ khóa như 'work visa', 'sponsorship' hoặc liên hệ bộ phận HR để được hỗ trợ."

Generation Failure Patterns

VINUNIVERSITY

Xung đột ngữ cảnh

2 tài liệu mâu thuẫn (VD: 12 ngày vs 14 ngày phép).

LLM bối rối, chọn bừa hoặc cộng gộp.

Fix: "Quay lại Context Resolution"

Suy diễn quá đà

Tài liệu: "Free ship đơn 500k ở HN". User hỏi "HCM thì sao?"

LLM tự suy: "HN được thì HCM chắc cũng được" → hallucination.

Fix: "Không tự suy luận điều kiện không được đề cập rõ ràng."

Bỏ qua constraints

Đã dặn trích dẫn [doc_id] nhưng model quên. Hay gặp với model nhỏ hoặc context dài.

Fix: Đặt rule quan trọng ở **cuối prompt** (gần "Answer:" nhất). Giảm `temperature = 0`.

In `formatted_context` ra log.
Context **có** đáp án → lỗi Generation (sửa prompt/model).
Context **không** có đáp án → lỗi Retrieval.

Agentic RAG

08

Cấp độ cao nhất: hệ thống không chỉ tìm kiếm thụ động mà chủ động suy luận để giải quyết vấn đề

Agentic RAG: Reasoning Loops

VINUNIVERSITY

Self-Query

Agent tự **tách câu hỏi phức tạp** thành các câu hỏi nhỏ để tìm kiếm nhiều lần, rồi tổng hợp kết quả.

Corrective RAG (C-RAG)

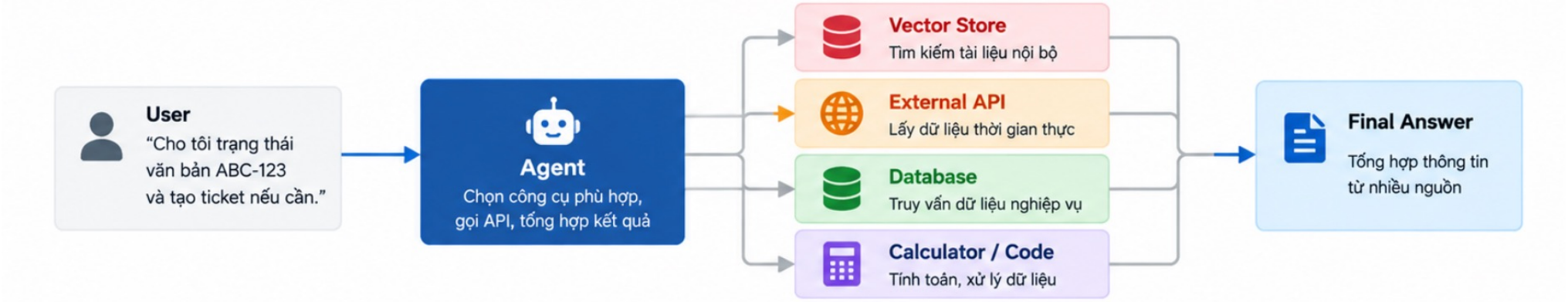
Agent tự **đánh giá chất lượng** tài liệu tìm được. Nếu tài liệu rác → tự động kích hoạt **tìm kiếm Web** hoặc báo lỗi.

Adaptive RAG

Agent tự **chọn chiến thuật**: câu hỏi dễ → trả lời ngay; câu hỏi khó → kích hoạt tìm kiếm chuyên sâu.

Lưu ý: Agentic RAG = RAG + reasoning loop. Agent không chỉ search 1 lần mà **lặp lại, tự sửa, tự quyết định** chiến thuật.

Agentic RAG: Tool Use



Vector Store (RAG)

- Gọi API (ví dụ: eDocman) để tìm trạng thái văn bản trong tài liệu nội bộ
- Trả về nội dung, quy trình, chính sách liên quan

External API

- Gọi API bên ngoài (ví dụ: JIRA) để lấy thông tin mới nhất
- Lấy dữ liệu thời gian thực (ticket, trạng thái hệ thống, ...)

Database

- Truy vấn cơ sở dữ liệu nghiệp vụ (ví dụ: thông tin nhân sự, khách hàng, đơn hàng, ...)
- Lấy dữ liệu có cấu trúc, chính xác

Calculator / Code Interpreter

- Thực hiện tính toán (tổng, chênh lệch, thống kê, ...)
- Chạy code, xử lý dữ liệu
- Hỗ trợ các tác vụ phân tích phức tạp

Ví dụ minh họa

User:
"Cho tôi trạng thái văn bản ABC-123 và tạo ticket nếu văn bản đang trễ hạn."

Agent (kế hoạch hành động):
1. Gọi eDocman API → kiểm tra trạng thái văn bản ABC-123
2. Nếu trễ hạn → gọi JIRA API → tạo ticket mới
3. Trả lời kết quả tổng hợp cho người dùng

Kết quả:
"Văn bản ABC-123 hiện đang trễ hạn 3 ngày. Tôi đã tạo ticket JIRA: RAG-4567 để xử lý. Bạn có thể xem chi tiết tại đây: ..."

RAG Evaluation

09

Không có evaluation thì không biết hệ thống đang tốt lên hay chỉ đang thay đổi cách viết câu trả lời

Vibe Check Là Không Đủ (Why Evaluate?)

VINUNIVERSITY

Kiểm thử thủ công

- Hỏi vài câu, thấy "ổn" → deploy
- Không phát hiện edge case
- Không biết regression khi đổi config
- Không có bằng chứng cho stakeholder

Evaluation có hệ thống

- Bộ test cố định, chạy lại được
- Phát hiện lỗi do Retriever hay Generator
- So sánh trước/sau khi thay đổi
- Báo cáo bằng số liệu cho PM/sếp

Lưu ý: "Cảm giác ổn" không phải metric. RAG cần **test set + scorecard + so sánh A/B** để biết thực sự đang tốt lên.

Xây Test Set: Bước Không Ai Muốn Làm Nhưng Bắt Buộc

VINUNIVERSITY

Mỗi test case gồm 3 phần

1. **question** — viết như user thật hỏi, kể cả sai chính tả
2. **ground_truth** — câu trả lời đúng (người viết)
3. **expected_chunks** — `doc_id` của chunk **phải** được retrieve

```
{``question'': ``Nghi phep bao nhieu ngay?'',
``ground_truth'': ``14 ngay/nam'',
``expected_chunks'': [``hr-leave-v3#s2'']}
```

Bao nhiêu là đủ? Lấy ở đâu?

15–20 câu là đủ để bắt regression. 50–100 cho production.

Nguồn tốt nhất, theo thứ tự:

- **Log câu hỏi thật** của user (vàng)
- Câu hỏi hay gặp từ team CS / HR
- Người có domain viết tay
- LLM sinh nháp → **người review lại**

Cố tình đưa vào các ca khó — đây mới là chỗ test set có giá trị

Câu hỏi có **mã lỗi / tên riêng** (bắt lỗi dense-only) · câu **multi-hop** (bắt lỗi thiếu decomposition) · câu mà **corpus không có đáp án** (phải abstain, không được bịa) · câu có **2 doc mâu thuẫn** (phải chọn bản mới)

Lưu ý: Test set do **LLM sinh 100%** và không ai review sẽ chỉ đo được "model có đồng ý với chính nó". Luôn cần người chốt `ground_truth`.

Case Study: Hybrid Search Lên Bàn Cân

VINUNIVERSITY

V1: Dense Only

- Lấy đúng chunk: **12/20 câu**
- Bỏ lỡ mã lỗi, tên riêng, số ticket
- Trả lời thiếu vì context không đủ

V2: Hybrid (BM25 + Vector)

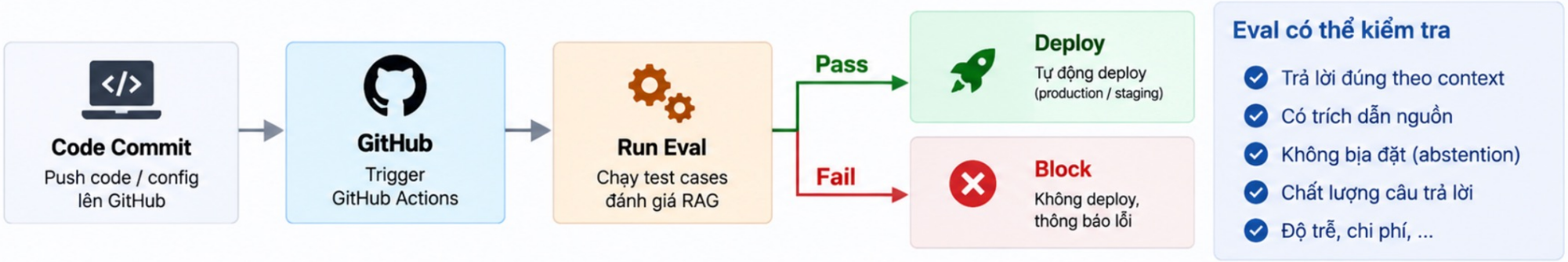
- Lấy đúng chunk: **18/20 câu**
- Bắt được exact term nhờ BM25
- Trả lời đầy đủ hơn vì context đủ

Số liệu minh hoạ cho một corpus có nhiều mã lỗi và tên riêng — mức tăng thật phụ thuộc corpus của bạn. Hãy đo trên test set của chính mình.

- Thêm BM25 thường là thay đổi **rẻ nhất** cho mức cải thiện recall lớn nhất — nên thử **trước** khi nghĩ tới rerank hay đổi model.

- Chi tiết về RAG Evaluation – Day 14

CI/CD Cho RAG Evaluation



1. Viết test case (ví dụ)

```
{
  "question": "Chính sách nghỉ phép
   của công ty là gì?",
  "expected_contains": [
    "12 ngày phép",
    "được phê duyệt bởi quản lý"
  ],
  "must_cite": true,
  "must_not_hallucinate": true
}
```

2. Chạy eval trong GitHub Actions

```
# .github/workflows/eval.yml
name: RAG Eval
on: [push]
jobs:
  eval:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run RAG evaluation
        run: python eval/run_eval.py
```

3. Điều kiện deploy

```
if eval_passed:
    deploy()
    notify("✅ Eval passed. Deployed!")
else:
    block_deploy()
    notify("❌ Eval failed. Check logs.")
```

4. Kết quả trên GitHub

RAG Eval ✓

✓ test_grounded_answer	12s
✓ test_citations	8s
✓ test_abstention	7s
✗ test_policy_update	15s

Lợi ích

- Đảm bảo chất lượng và an toàn của RAG trước khi triển khai.
- Phát hiện sớm lỗi (hallucination, sai nguồn, hiệu năng kém).
- Tự động hoá quy trình kiểm thử → deploy, **giúp vận hành ổn định và tin cậy.**

Index Freshness: Câu Trả Lời Đúng Của Tuần Trước

VINUNIVERSITY

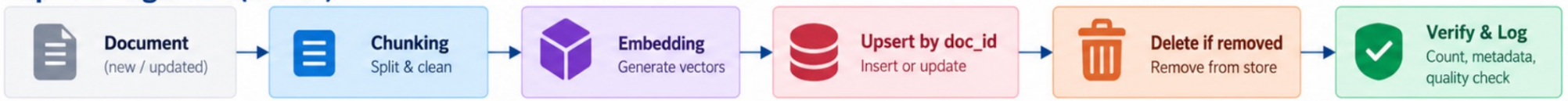
Vấn đề thường gặp

1. **Không metric nào được truy hồi**
   Ví dụ: Policy đổi ngày 01/08 nhưng kết quả vẫn trả về bản cũ.
2. **Metric tưởng đúng nhưng sai thực tế**
   Câu trả lời bám context, nhưng context cũ, thiếu hoặc đã lỗi thời.
3. **Thiếu kiểm soát chất lượng dữ liệu**
   Không phát hiện được bản trùng, thiếu metadata, hoặc chunk bị lỗi khi index.

Ba việc phải làm

1. **Upsert theo doc_id**
   - Re-index và ghi đè tài liệu theo doc_id (không tạo bản thứ hai).
   - Nếu không, các version cũ và mới sẽ cùng tồn tại trong index.
2. **Xoá thật khi thu hồi**
   - Khi tài liệu bị thu hồi (ví dụ: đã có ở Day 07), phải delete khỏi vector store
   - (bao gồm embeddings, metadata). Không chỉ đánh dấu.
3. **Hiển thị ngày/phiên bản trên UI**
   - Cho người dùng thấy metadata (ví dụ: "cập nhật ngày 10/01/2026") để họ tự xác minh dữ liệu.

Pipeline ingestion (đề xuất)



Ví dụ minh họa

Tài liệu: HR Policy
Cập nhật ngày: 10/01/2026
doc_id: hr_policy_v3

- Hệ thống upsert theo doc_id = hr_policy, ghi đè phiên bản cũ.
- Nếu tài liệu bị thu hồi (đã có ở Day 07) → delete hoàn toàn khỏi vector store.
- Trên UI hiển thị: "HR Policy (cập nhật 10/01/2026)".

Kết quả

- ☑ Truy hồi đúng phiên bản mới nhất
- ☑ Không còn dữ liệu cũ gây nhiễu
- ☑ Người dùng có thể kiểm tra ngày cập nhật
- ☑ Dễ debug và kiểm soát chất lượng dữ liệu

Lưu ý: Ingestion là một pipeline chạy lặp, không phải script chạy một lần. Metadata (ngày cập nhật, version, nguồn, trạng thái) là chìa khóa để phát hiện và xử lý lỗi.

ROI Của RAG (Chi Phí vs. Chất Lượng)

VINUNIVERSITY

Ví dụ giả định: Thêm Cross-encoder Reranker

- Chất lượng câu trả lời tăng **nhẹ**
- Latency tăng từ 1s → **4s**
- Chi phí server tăng **gấp đôi**

Con số minh hoạ để tập cân nhắc trade-off — không phải bench-mark.

Bài toán của kỹ sư trưởng

5% độ chính xác đó có đáng giá với:

- Trải nghiệm chậm chạp của người dùng?
- $500/tháng chi phí thêm?
- +3s latency mỗi request?

Lưu ý: Không phải kỹ thuật nào cũng đáng thêm. Mỗi cải tiến cần cân nhắc **quality gain vs. latency vs. cost**.

Khi nào cần RAG ?

In-Context
(Knowledge rất nhỏ)

Quy mô knowledge
Rất nhỏ
(vài câu đến vài trang)

Khi nào dùng?
- Có thể đưa toàn bộ knowledge trực tiếp vào prompt.
- Thường dùng cho tác vụ ad-hoc hoặc ít thay đổi.
- Không cần hệ thống phức tạp.

Ví dụ
Có 3 trang policy về nghỉ phép. Đưa toàn bộ vào prompt và hỏi: "Nhân viên được nghỉ phép bao nhiêu ngày?"

CAG
(Knowledge nhỏ)

Quy mô knowledge
Nhỏ
(vài chục trang, có thể cache)

Khi nào dùng?
- Có thể preload/cache toàn bộ knowledge.
- Thường được hỏi nhiều lần, ít thay đổi.
- Giảm chi phí retrieval, tăng tốc độ trả lời.

Ví dụ
Employee Handbook ~ 50 trang, ít thay đổi. Cache toàn bộ và dùng để trả lời nhiều câu hỏi về chính sách công ty.

RAG
(Knowledge lớn)

Quy mô knowledge
Lớn
(hàng trăm đến hàng triệu tài liệu)

Khi nào dùng?
- Không thể/không nên đưa toàn bộ vào context.
- Cần tìm kiếm và lấy phần liên quan (retrieve).
- Thích hợp khi tài liệu phân tán, thay đổi thường xuyên.
- Thường cần trích dẫn nguồn (citation).

Ví dụ
50.000 SOP, policy, luật, tài liệu kỹ thuật. Retrieve Top-K chunks liên quan đến câu hỏi và trả lời kèm trích dẫn.

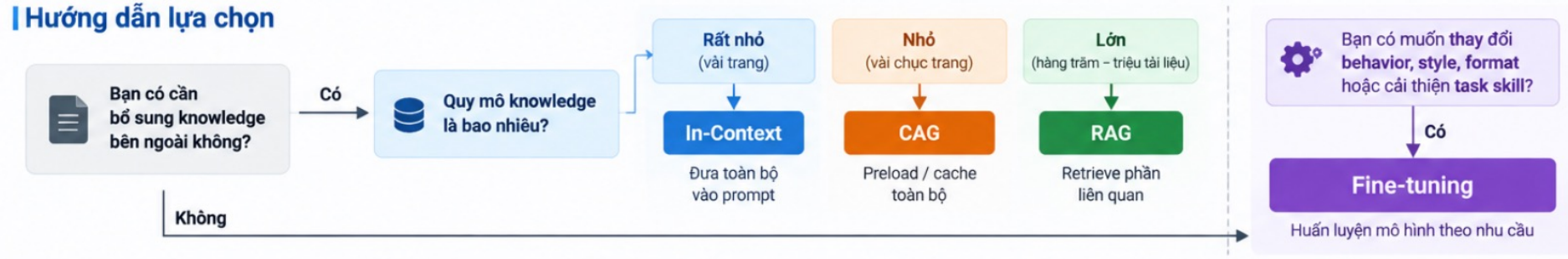
Fine-tuning
(Thay đổi behavior)

Mục tiêu chính
Không phụ thuộc vào quy mô knowledge, mà nhằm thay đổi cách model hành xử.

Khi nào dùng?
- Cần mô hình tuân theo format, style, tone cụ thể.
- Cải thiện khả năng xử lý task đặc thù (domain skill).
- Giảm phụ thuộc vào prompt dài, ổn định behavior.

Ví dụ
Fine-tune model để luôn trả lời theo format: Policy → Eligibility → Action → Contact, với giọng điệu thân thiện, chuyên nghiệp.

Hướng dẫn lựa chọn



Hands-on 8

11

Nâng cấp pipeline Day 07 thành RAG Service có đo lường, xây KB thứ ba từ dữ liệu thật, rồi tự kiểm chứng khi nào RAG *không* phải đáp án

Lab #8

VINUNIVERSITY

Mục tiêu

- Nâng cấp KnowledgeBase của Day 07 thành RAG Service đầy đủ (R–A–G).
- Xây 3 KB: Tin tức, Luật pháp, Sản phẩm thẻ từ dữ liệu thực tế.
- Thực hành ingestion – retrieval – evaluation – serving.
- Khám phá hạn chế của RAG hiện tại qua bài toán liên kết Tin tức ↔ Luật pháp.
- Xây dựng data compilation pipeline từ dữ liệu crawl cho Sản phẩm thẻ.

3 Knowledge Base và mục tiêu học tập

1. News – Tin tức

Đặc điểm dữ liệu

- Văn xuôi, cập nhật liên tục
- Nhiều chủ đề, cách diễn đạt đa dạng

Mục tiêu học tập

- Retrieval trên dữ liệu văn xuôi
- Dense/semantic retrieval phù hợp với câu hỏi tự nhiên, paraphrase

2. Law – Luật pháp

Đặc điểm dữ liệu

- Cấu trúc chặt chẽ (Điều, Khoản)
- Thuật ngữ pháp lý, tính chính xác cao, ít thay đổi

Mục tiêu học tập

- Chunking theo cấu trúc
- Kết hợp keyword/sparse retrieval
- Sử dụng metadata (Điều, Khoản, ...)

3. Cards – Sản phẩm thẻ

Đặc điểm dữ liệu

- Thu thập từ dữ liệu crawl thực tế
- Nhiều định dạng (HTML, PDF, ...)
- Nhiều sản phẩm, thuộc tính đa dạng

Mục tiêu học tập

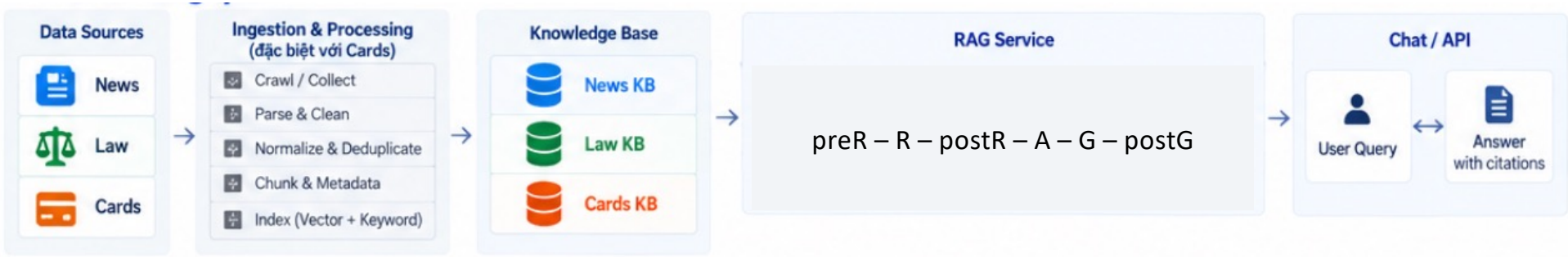
- Xây dựng data compilation pipeline
- Parse, clean, normalize, deduplicate
- Chunk và gắn metadata phù hợp

Ví dụ câu hỏi

"Miu Lê sử dụng ma tuý ở đâu và vi phạm bộ luật hình sự nào?"

Ý nghĩa

Câu hỏi cần kết hợp thông tin từ tin tức và văn bản pháp luật, cho thấy hạn chế của RAG hiện tại trong bài toán liên kết đa nguồn.



Lab 8: Ba Pha

VINUNIVERSITY

Phase 1 — Cá nhân: build pipeline

1. `retriever.py` — thêm BM25 song song dense, gộp bằng **RRF**
2. `augment.py` — đóng gói context có `doc_id` + ngày, reorder, giữ context ≤ 60% budget
3. `generate.py` — grounded prompt, **bắt buộc citation**, abstain khi thiếu chứng cứ
4. Giữ lại đường **V1 dense-only** để so sánh

Phase 2 — Nhóm: đo và tranh luận

1. Nhóm cùng viết **15–20 test case** trên corpus Day 07 (có ca khó: mã lỗi, multi-hop, không có đáp án, mâu thuẫn)
2. Mỗi người chạy **V1 vs V2** trên cùng test set
3. Điền **scorecard**: mỗi test case V1/V2 **đạt** hay **không đạt**
4. Với ca **không đạt**: chỉ ra lỗi thuộc R / A / G / Indexing
5. Demo: 1 ca V2 thắng rõ, 1 ca V2 **không** thắng

V2 phải chứng minh bằng số, không phải "em thấy nó hay hơn".
V2 không hơn V1 cũng là kết quả hợp lệ.

Phase 3 — Về nhà: KB `cards` + in-context (Exercise 3.4)

A. Crawl thẻ **MSB, TCB, VPB, VIB** → **B.** Build KB `cards` → **C.** Hybrid search 5 loại query, ghi **cảm nhận** α → **D.** Tổng hợp `product_wiki.md` → **E.** So **in-context** với RAG

Deliverable Và Assessment

Nộp bài

- `src/` — hybrid retriever, augment, generate
- `eval/testset.jsonl` + `eval/scorecard.md`
- `cards_raw.json` + `product_wiki.md`
- `report/REPORT.md` — 1 báo cáo / sinh viên

Phần	Điểm
Cá nhân (pipeline R–A–G)	40
Cá nhân (eval + phân tích)	15
Nhóm (test set + so sánh)	20
KB `cards` (crawl + hybrid)	15
Wiki + so sánh in-context	10
Tổng	**100**

6 góc nhìn tự đánh giá

1. **Retrieval** — hybrid có bắt được ca dense-only bỏ lỡ?
2. **Grounding** — mọi claim đều có citation truy được?
3. **Abstention** — có từ chối đúng lúc, không bịa?
4. **Token Discipline** — context có tôn trọng budget 60%?
5. **Diagnosis** — có chỉ đúng lỗi thuộc khối nào?
6. **Architecture** — có **giải thích được** vì sao chọn RAG hay in-context cho từng KB?

Checklist Trước Khi Nộp

- ☑ **Hybrid search** chạy được cả dense và BM25, gộp bằng RRF
- ☑ **Citation** xuất hiện trong mọi câu trả lời, `doc_id` truy được về chunk gốc
- ☑ **Abstention** hoạt động — có ít nhất 1 test case model từ chối đúng
- ☑ **Test set** ≥ 15 case, có đủ 4 loại ca khó
- ☑ **Scorecard** có số của cả V1 và V2 trên cùng test set
- ☑ **Nguồn** là text/HTML — **không** dùng bản scan qua OCR
- ☑ **KB** `cards` đủ 4 ngân hàng, mỗi thẻ có `source_url` + `crawled_at`
- ☑ `product_wiki.md` đã chuẩn hoá đơn vị phí/lãi suất, có bảng so sánh theo hạng
- ☑ **Report** nêu 1 ca V2 thắng, 1 ca V2 không thắng, và kết luận RAG/in-context cho cả 3 KB

Tài Liệu Tham Khảo

1. Lewis et al. (2020), *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*.
2. OpenAI Docs, *Retrieval Guide* và *File Search Guide*.
3. LangChain, *RAG from Scratch* notebooks.
4. LlamaIndex Docs, *Starter Example*.
5. Cohere Docs, *Rerank overview*.
6. Liu et al. (2023), *Lost in the Middle: How Language Models Use Long Contexts*.
7. Yan et al. (2024), *Corrective Retrieval Augmented Generation (C-RAG)*.

Hỏi & Đáp

Bạn đang thiếu model mạnh hơn, hay đang thiếu một pipeline retrieval, augmentation, và evaluation đủ kỷ luật?